

Part 1: Variational Autoencoder on MNIST — Report

All experiments are implemented in `CW2-Part-1.ipynb`. The notebook runs end-to-end on CUDA, MPS, or CPU with `SEED=60027`, and saves figures to `figures/part1/`.

Task 1 (25%): Baseline VAE and training. The encoder maps flattened 784-dimensional MNIST inputs through a 400-unit ReLU hidden layer and outputs $(\mu, \log \sigma^2) \in \mathbb{R}^{20}$ for a diagonal Gaussian posterior; the decoder mirrors this with a sigmoid output giving a Bernoulli pixel distribution. Total parameters: 652,824. The training objective is the β -ELBO with $\beta = 1$:

$$\mathcal{L} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[-\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction (BCE)}} + \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel \mathcal{N}(\mathbf{0}, I))}_{\text{KL divergence}},$$

where the reconstruction term is the binary cross-entropy summed over pixels and averaged over the batch, and the KL term admits the closed form

$$D_{\text{KL}} = -\frac{1}{2N} \sum_{n=1}^N \sum_{j=1}^{20} (1 + \log \sigma_{nj}^2 - \mu_{nj}^2 - \sigma_{nj}^2),$$

where N is the batch size. Gradients with respect to ϕ flow through the discrete sampling step via the reparameterisation trick: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I)$, with $\boldsymbol{\sigma} = \exp(0.5 \log \boldsymbol{\sigma}^2)$. The encoder outputs $\log \boldsymbol{\sigma}^2$ rather than $\boldsymbol{\sigma}^2$ directly, ensuring the variance remains positive for any real-valued linear output. Training uses Adam (lr = 10^{-3} , $\beta_1=0.9$, $\beta_2=0.999$; no scheduler or gradient clipping), batch size 128, for 30 epochs ($\approx 14,000$ gradient steps).

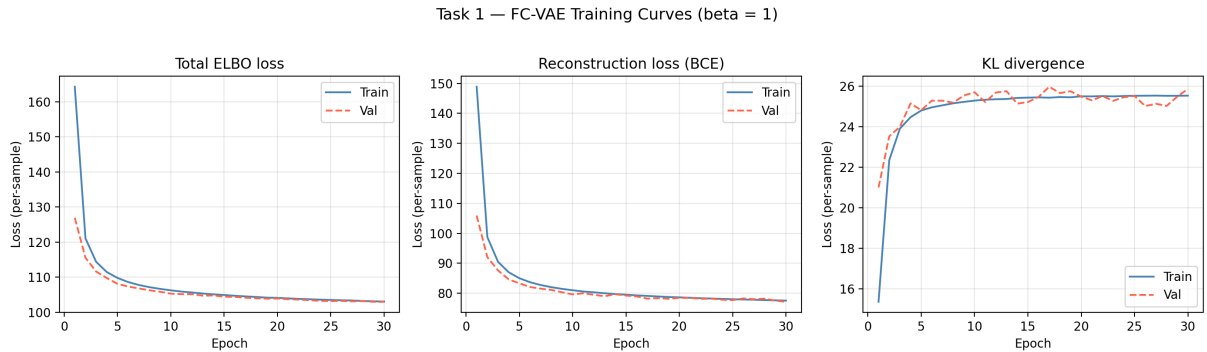


Figure 1: Task 1 — FC-VAE training and validation loss curves. *Left:* total ELBO. *Centre:* reconstruction (BCE per sample). *Right:* KL divergence.

Figure 1 and Table 1 show the training progression. The total loss drops sharply from 164.24 at epoch 1 to 109.81 at epoch 5 (-54.43 nats), driven almost entirely by the reconstruction term ($148.89 \rightarrow 85.01$, -63.88 nats), as the decoder first learns coarse digit structure. Beyond epoch 5 both curves transition to a slow, flat decline: the per-five-epoch improvement shrinks from 3.53 (epochs 5–10) to 0.46 (epochs 25–30). The KL term follows an opposite trajectory: it rises from 15.35 at epoch 1 to a plateau of 25.54 at epochs 25–30 and holds steady. The initial low KL reflects the encoder posterior remaining close to the prior before any useful representation has formed; the subsequent rise indicates the encoder diverging from $\mathcal{N}(\mathbf{0}, I)$ to encode class-relevant information. The plateau at ≈ 25.5 nats signals convergence to a stable

compression–reconstruction balance. Crucially, no KL collapse ($\text{KL} \rightarrow 0$) occurs, so the model does not show posterior collapse at the aggregate level. The train/validation gap is negligible throughout ($\Delta = 0.07$ at epoch 30), suggesting little evidence of overfitting within the first 30 epochs.

Table 1: FC-VAE training log (selected epochs, $\beta = 1$, $z_{\text{dim}} = 20$).

Epoch	Train total	Train recon	Train KL	Val total
1	164.24	148.89	15.35	126.92
5	109.81	85.01	24.81	108.13
10	106.28	80.98	25.30	105.33
15	104.95	79.51	25.44	104.45
20	104.13	78.62	25.51	103.95
25	103.54	78.00	25.54	103.22
30	103.08	77.54	25.54	103.01

Justification for 30 epochs. The five-epoch absolute improvement in total loss is 54.43 (epochs 1–5), 3.53 (5–10), 1.33 (10–15), 0.82 (15–20), 0.59 (20–25), and 0.46 (25–30). The diminishing-returns pattern shows that training has effectively converged by epoch 25. Stopping at epoch 15 would sacrifice 1.87 nats; the final 5-epoch gain is already only 0.46 nats. Thirty epochs is therefore a reasonable stopping point for this architecture.

Task 2 (20%): Reconstruction vs generation. Using the trained FC-VAE we generate three sets of 64 images via two mechanically distinct paths. At test time no gradient is required, so sampling uses the same reparameterisation expression ($\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$) but inside `torch.no_grad()`. *Path 1 – posterior-conditioned* ($x \rightarrow \text{encode} \rightarrow z \sim q_\phi(\mathbf{z}|\mathbf{x}) \rightarrow \text{decode}$): used for both (a) *reconstructions* and (b) *aggregated posterior samples*. Both panels share an identical pipeline; they differ only in interpretation. *Path 2 – prior* ($\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I) \rightarrow \text{decode}$): used for (c) *prior samples*, the only genuinely distinct generation path.

Figure 2 shows a clear quality hierarchy. The two posterior-conditioned panels (reconstructions and aggregated posterior) follow the same pipeline $x \rightarrow q_\phi(z|x) \rightarrow \text{decode}$ and are therefore visually similar by construction; any residual difference is sampling noise, not a change in mechanism. Both panels yield sharp, digit-recognisable images, as the latent codes lie in the high-density region of the learned posterior. Prior samples are the least structured: many are blurry, class-ambiguous, or morphologically inconsistent despite using the same decoder. The root cause is the *prior–posterior mismatch*: the encoder maps each class to a distinct region in $\mathbb{R}^{20}$ (though with inter-class overlap, as quantified in Task 3), so the aggregated posterior $q_A(\mathbf{z}) = \int q_\phi(\mathbf{z}|\mathbf{x}) p_{\text{data}}(\mathbf{x}) d\mathbf{x}$ is a class-structured distribution occupying a proper subset of the prior’s support. The isotropic prior $\mathcal{N}(\mathbf{0}, I)$ places substantial mass in regions where the decoder is poorly trained, producing the observed blurry and ambiguous outputs. This mismatch is quantified in Task 3.

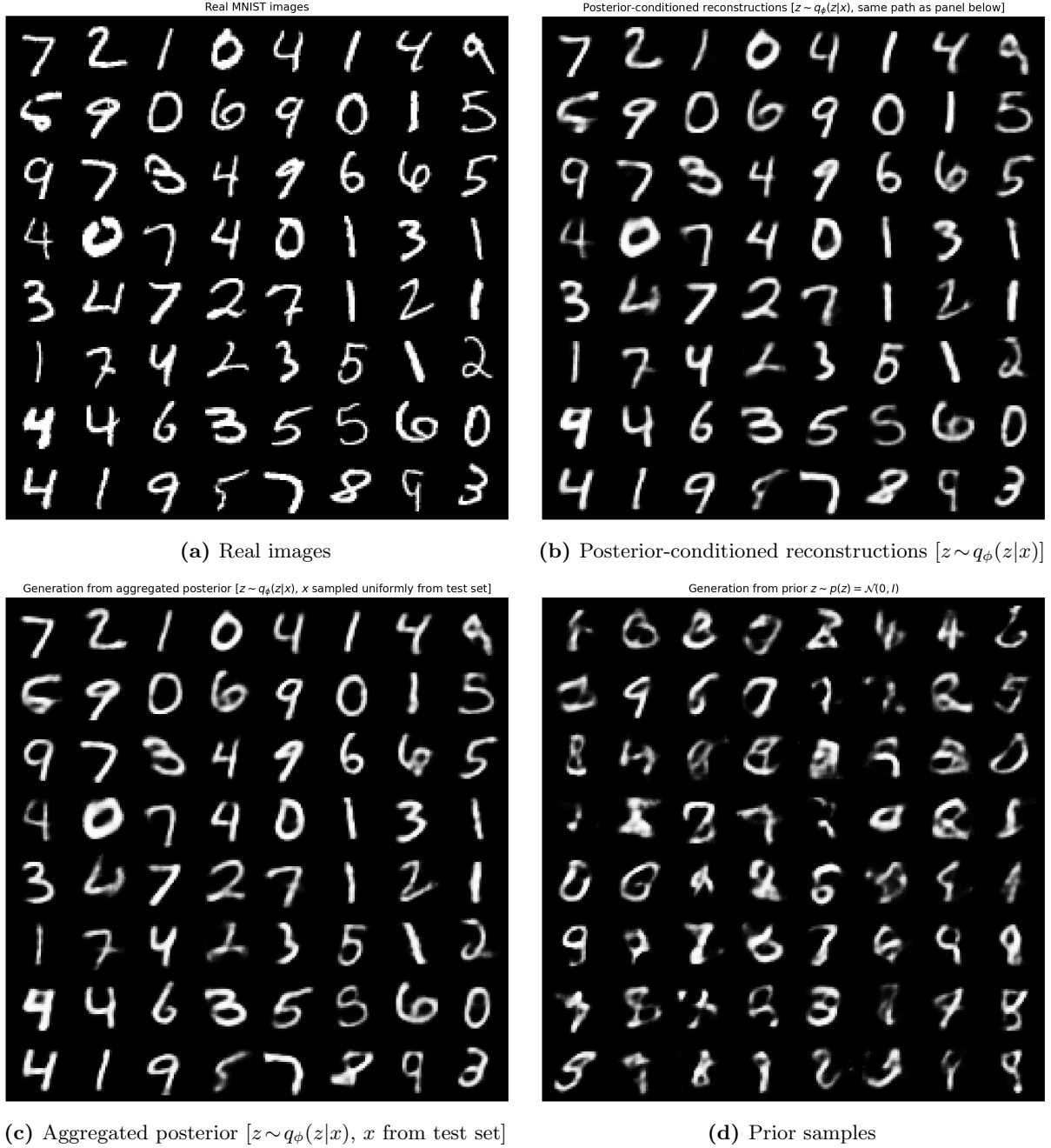


Figure 2: Task 2 — Comparison of reconstruction and generation quality (8 × 8 grids, 64 images each).

Task 3 (20%): Comparing prior and aggregated posterior. We assess the gap between $q_A(\mathbf{z})$ and $p(\mathbf{z})$ using five complementary analyses applied to 10,000 test-set encoder samples.

What is new beyond lecture material. The brief requires PCA comparison and Wasserstein distance. In addition we include: (i) a class-centroid plot with 1.5σ covariance ellipses in PCA space, making cluster shape and overlap explicit; (ii) non-linear 2D projections (t-SNE, UMAP) to reveal local neighbourhood structure not visible in a linear projection; (iii) quantitative projection diagnostics (silhouette score and trustworthiness) to compare projection fidelity objectively; (iv) latent traversal along the four most active latent dimensions, showing how the decoder responds to systematic variation in individual z coordinates.

PCA projection and class centroid structure (Figures 3 and 4). PCA is fitted on the 10,000 encoder means μ (not noisy samples $z \sim q(z|x)$), giving a stable view of class geometry. The left panel of Figure 3 reveals partial class-to-class structure: some groupings are visible, but

substantial overlap persists, consistent with PC 1 (7.8%) and PC 2 (7.1%) together accounting for only $\approx 15\%$ of the total variance in the 20-D latent space. The right panel projects sampled $q_A(z)$ and $p(z)$ with the same axes: the prior (blue) occupies a wider, more isotropic spread, while the aggregated posterior (red) is noticeably more concentrated, confirming the prior-posterior mismatch. Figure 4 quantifies cluster shape: 1.5σ covariance ellipses reveal that some digit pairs (e.g. 4 and 9) overlap in the top-2 PCA directions, while others (e.g. 0 and 1) are well-separated. *Limitation of PCA*: being a global linear method, it compresses 20 dimensions to 2 and may merge clusters separated in orthogonal directions.

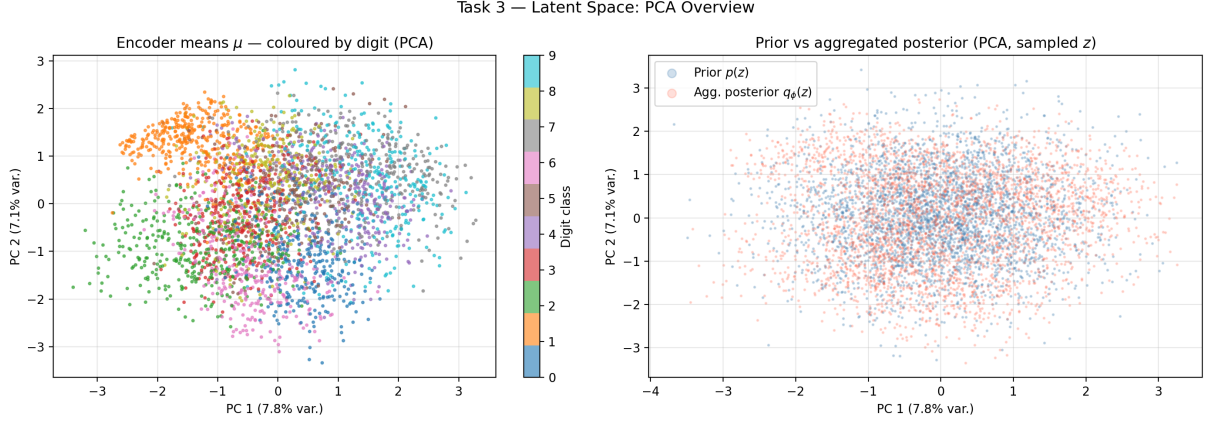


Figure 3: Task 3 — 2D PCA of the latent space. *Left:* encoder means μ coloured by digit class. *Right:* sampled aggregated posterior $q_A(z)$ (red) vs isotropic prior $p(z)$ (blue).

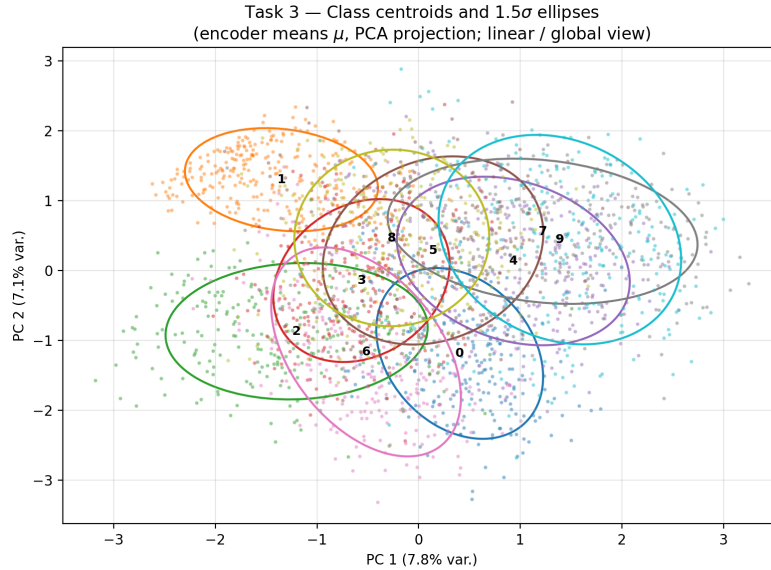


Figure 4: Task 3 — Class centroids (digit labels) and 1.5σ covariance ellipses in the 2D PCA space of encoder means μ .

Non-linear projections and projection diagnostics (Figure 5). PCA may not expose fine class separation in 2D when the latent manifold is non-linear or when relevant variation lies in minor principal directions. We apply t-SNE (perplexity = 35) and UMAP ($k = 20$ neighbours, min-dist = 0.1) to the same $n = 1,200$ encoder-mean subsample. Both methods reveal much cleaner cluster separation than PCA on the same subset. However, *neither t-SNE nor UMAP preserves global metric geometry*: inter-cluster distances in their 2D outputs are not interpretable as true latent distances. To compare projections objectively we report two statistics: *trustworthiness* at

$k = 15$ neighbours measures the fraction of the 15 nearest neighbours in the embedding that were also among the 15 nearest neighbours in the original 20-D space (values near 1 indicate faithful local structure); *silhouette score* in the 2D embedding quantifies class compactness relative to separation (negative values indicate overlapping clusters in that 2D view).

Projection	Silhouette (2D)	Trustworthiness@15
PCA	−0.0662	0.6885
t-SNE	0.2786	0.9512
UMAP	0.3707	0.9385

The negative PCA silhouette confirms that 2D PCA is insufficient to separate all 10 classes: clusters overlap in the two leading principal components. t-SNE and UMAP both achieve positive silhouette scores and high trustworthiness (> 0.93), confirming that the 20-D latent space *does* contain well-separated local class neighbourhoods that require non-linear projection to be visible in 2D. The 20-D silhouette score on encoder means (full $n = 10,000$) is reported separately as a measure of cluster quality in the original latent space, unaffected by projection distortions.

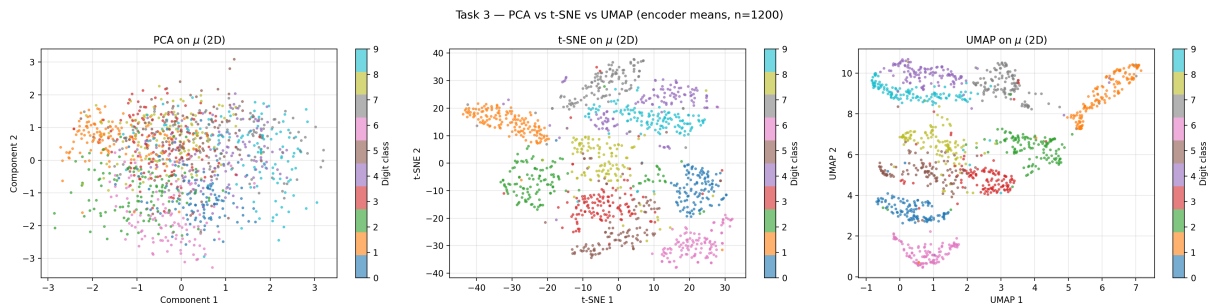


Figure 5: Task 3 — PCA (left), t-SNE (centre), and UMAP (right) projections of encoder means μ ($n = 1,200$, coloured by digit class). t-SNE and UMAP reveal cleaner local cluster structure than PCA; inter-cluster distances in t-SNE/UMAP are not interpretable.

Latent traversal (Figure 6). To confirm that learned latent dimensions are semantically active, we traverse the four dimensions with the highest marginal standard deviation in the aggregated posterior (dimensions $\{1, 17, 11, 8\}$) from -3 to $+3$ around the encoder mean of a reference test image. Smooth, structured changes in digit style, stroke thickness, and orientation are observed across these dimensions, confirming that the decoder has assigned specific visual factors to individual latent coordinates rather than distributing variation uniformly. This also explains why posterior-conditioned sampling in Task 2 produces coherent digits: the latent code activates only the well-trained decoder regions.

Per-dimension statistics (Figure 7). The posterior means span $[-0.102, 0.119]$ across all 20 dimensions, close to the prior mean of 0, as expected from KL regularisation. The posterior variances range from 0.952 to 1.131, consistently deviating from the prior variance of 1. Dimensions with variance noticeably below 1 have been compressed toward the prior and encode little information; those above 1 are more active and carry digit-discriminative variation. This heterogeneity confirms that the 20 dimensions are not equally utilised, even though no KL collapse is observed.

Wasserstein distance (Figure 8). We compute the 1D Wasserstein distance W_1 for each latent dimension independently. This is a simple marginal diagnostic rather than a direct estimate of the full 20D transport distance, but it is still useful here because the KL objective encourages approximately factorised latent codes and the per-dimension statistics already show where mismatch concentrates. The results are $W_1^{\min} = 0.0193$, $W_1^{\max} = 0.1208$, $\bar{W}_1 = 0.0720$. To calibrate: for two unit-variance Gaussians separated by δ in mean, $W_1 = \delta$. A value $\bar{W}_1 \approx 0.07$ therefore corresponds to a sub-unit mean shift per dimension; small in absolute terms but

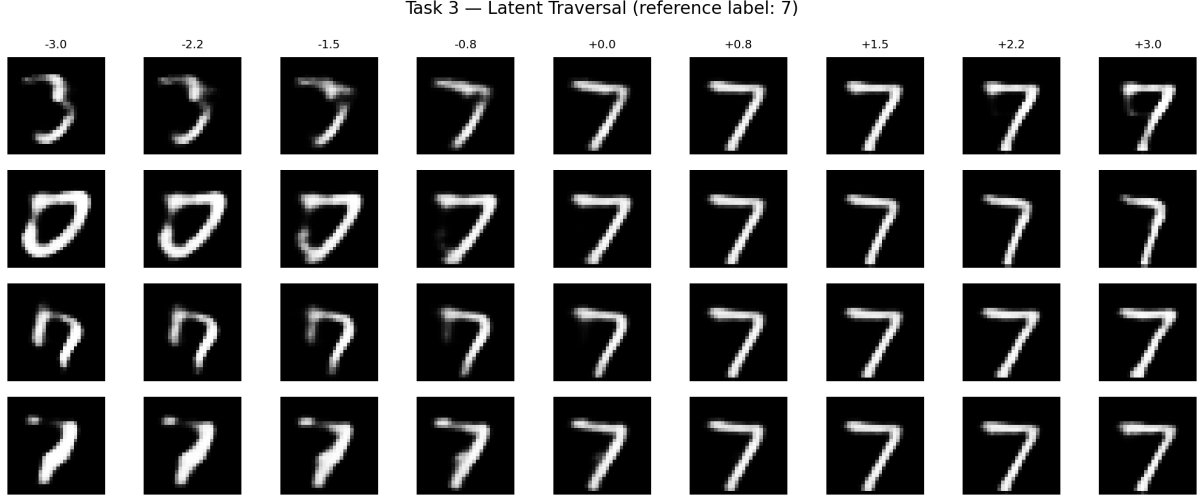


Figure 6: Task 3 — Latent traversal along the four most active dimensions ($\{1, 17, 11, 8\}$, ranked by posterior std). Columns sweep z_d from -3.0 to $+3.0$; all other coordinates are fixed at the reference code.

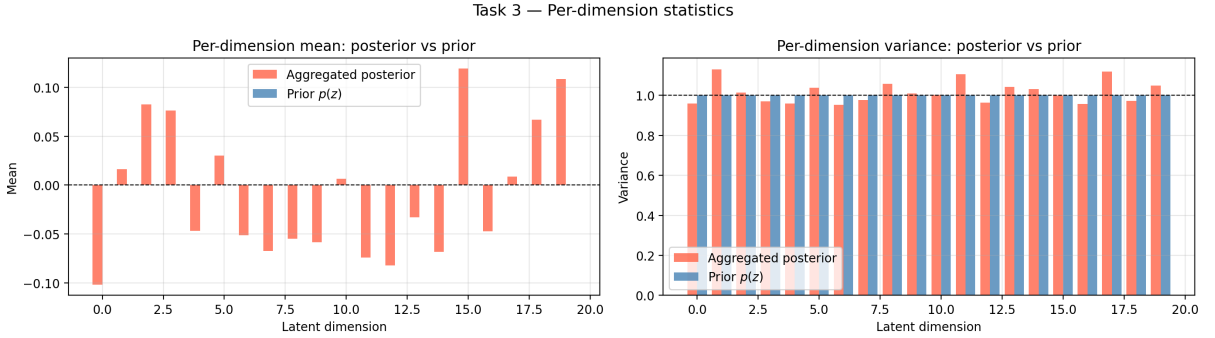


Figure 7: Task 3 — Per-dimension marginal means (left) and variances (right). Red: aggregated posterior. Blue: prior ($\mu = 0$, $\sigma^2 = 1$, shown as dashed reference lines).

non-negligible relative to the unit latent scale. A value $W_1 < 0.02$ per dimension would indicate near-perfect alignment (only the best-matched dimension achieves this); a value $W_1 > 0.5$ would indicate a severe mismatch. The observed range confirms a *moderate but real* mismatch, consistent with the visual evidence of generation quality degradation in Task 2.

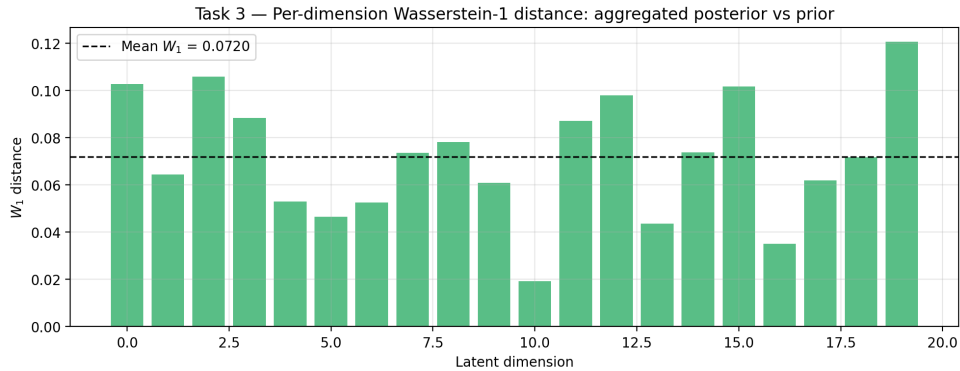


Figure 8: Task 3 — Per-dimension W_1 distance between the aggregated posterior and the prior. The dashed line marks the mean $\bar{W}_1 = 0.0720$.

Task 4 (20%): Reconstruction and regularisation (β -VAE). We introduce $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0\}$ multiplying the KL term and train each model for 20 epochs.

Theoretical effect of β . The KL penalty measures the information cost of encoding $\mathbf{x}$ in $\mathbf{z}$ beyond what the prior already provides. Scaling by β adjusts this cost: $\beta < 1$ allows the posterior to deviate strongly from the prior, letting each class occupy a wider, less constrained region in latent space and enabling lower reconstruction loss, but at the expense of a large prior–posterior mismatch that degrades unconditional generation. $\beta > 1$ penalises any deviation from $\mathcal{N}(\mathbf{0}, I)$, forcing posterior means toward $\mathbf{0}$ and variances toward 1; the aggregated posterior more closely matches the prior and generation improves, but the encoder can no longer retain as much information about $\mathbf{x}$, leading to blurrier reconstructions. At the limit $\beta \rightarrow \infty$ the KL collapses to zero, the encoder ignores the input, and the decoder degenerates into an unconditional prior sampler.

Table 2: β -VAE validation results (epoch 20). Active dims: number of latent dimensions with mean $\text{KL}_j > 0.01$ (threshold visible in Figure 12).

β	Val recon	Val KL	Val total	$\Delta\text{recon vs } \beta = 1$	Active dims
0.1	67.17	54.25	72.60	−11.29	20
0.5	72.05	34.32	89.21	−6.41	20
1.0	78.46	25.49	103.95	—	20
2.0	89.94	16.84	123.62	+11.48	18
5.0	116.26	7.94	155.96	+37.80	11
10.0	147.56	3.55	183.06	+69.10	6
20.0	192.05	0.59	203.85	+113.59	2

Empirical observations. Table 2 and Figures 9–11 confirm the theoretical predictions. Reconstruction loss increases and raw KL decreases monotonically with β across all seven settings, with the changes roughly symmetric on a log scale.

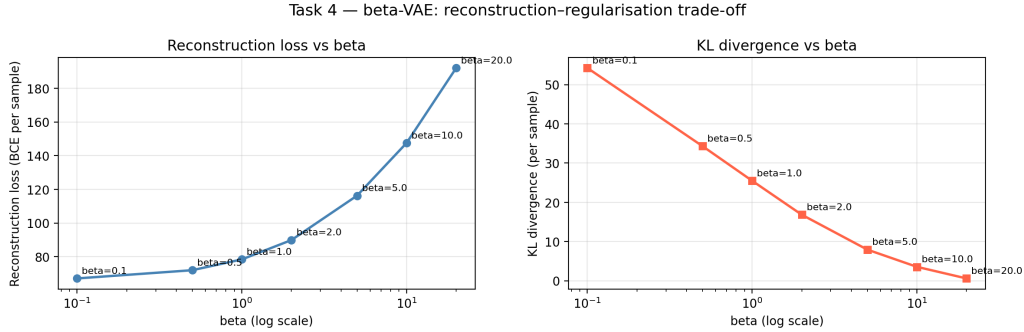


Figure 9: Task 4 — Validation reconstruction loss (left) and raw KL divergence (right) vs β on a log scale, for all seven $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0\}$. Reconstruction increases and KL decreases monotonically.

Prior-sample quality (Figure 10). At $\beta = 0.1$, prior samples are sharp in stroke but often class-ambiguous or erratic: with all 20 dimensions active and $\text{KL} = 54.25$, the aggregated posterior is severely misaligned with $\mathcal{N}(\mathbf{0}, I)$, so samples from the prior frequently land in poorly trained inter-cluster regions, producing confused digit shapes. At $\beta = 1.0$ the digits are clearer and more recognisable. At $\beta = 2.0$, prior samples are visually the cleanest across the sweep: digits are well-formed and class-consistent, reflecting improved prior–posterior alignment (18 active dimensions, reconstruction $\text{BCE} = 89.94$, +11.48 over $\beta = 1$). At $\beta = 5.0$, prior-sample quality begins to decline: strokes appear softer and less crisp, and visual diversity is somewhat reduced, consistent with 9 of 20 dimensions having collapsed (11 active). At $\beta = 10.0$ the degradation is clearly visible: digits are noticeably blurred and stroke detail is largely lost, consistent with only 6 active dimensions and reconstruction BCE of 147.56 (+69.10 over $\beta = 1$). At $\beta = 20.0$, the outputs are heavily degraded: strokes are smeared, contrast is very low, and strong visual

repetition suggests the decoder has collapsed to a small number of near-average outputs. With only 2 active dimensions and $KL = 0.59$, this is consistent with near-complete posterior collapse: the encoder effectively ignores the input and the decoder acts as an unconditional prior sampler.

Latent space structure (Figure 11). At $\beta = 0.1$, the latent codes span a wide range ($\approx [-4, 4]$ on the first PC) with some class groupings visible amid substantial overlap. As β increases, the distribution contracts monotonically toward the origin: at $\beta = 5.0$ the range narrows to $\approx [-2, 2]$ and class-wise separation diminishes. At $\beta = 10.0$ and $\beta = 20.0$ the contraction continues further ($\approx [-2, 2]$ and $\approx [-1.5, 1.5]$ respectively) and the point cloud becomes approximately isotropic near the origin, with nearly no class-discriminative structure visible, consistent with the encoder having collapsed most dimensions to the prior.

Per-dimension KL diagnostics (Figure 12). At $\beta = 0.1$, per-dimension KL values KL_j are high and roughly uniform (≈ 2.5 per dimension), indicating full utilisation of all 20 latent coordinates. As β increases, progressively more dimensions fall below the active threshold ($KL_j > 0.01$): the active count is 20 for $\beta \leq 1$, then 18, 11, 6, and 2 at $\beta = 2, 5, 10, 20$ respectively (right panel, Figure 12). At $\beta = 20$, the yellow line in the left panel lies at essentially zero across all 20 dimensions, confirming near-complete collapse.

Among the tested settings, $\beta = 2.0$ gives the best qualitative trade-off between prior-sample quality and reconstruction fidelity. A moderate $\beta > 1$ *does* improve unconditional generation quality relative to the standard VAE (fewer inter-cluster prior samples, cleaner digit strokes), but beyond $\beta \approx 5$ the benefit reverses as posterior collapse reduces both diversity and sharpness. It is important to keep the two quality metrics distinct: reconstruction quality (sampling from $q_\phi(z|x)$) degrades monotonically with β , while unconditional generation quality (sampling from $p(z)$) peaks at intermediate β before declining under over-regularisation.

Task 4 — Prior samples for different beta values



Figure 10: Task 4 — Prior samples ($\mathbf{z} \sim p(\mathbf{z})$, same fixed seed for all models) decoded by each β -VAE. Rows (top to bottom): $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0\}$. Quality peaks around $\beta = 2$ and degrades progressively at higher β , with near-complete collapse visible at $\beta = 20$.

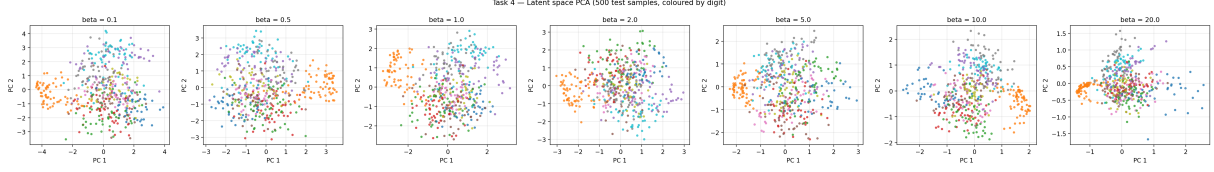


Figure 11: Task 4 — 2D PCA of the latent space (500 test images, coloured by digit class) for all seven $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0\}$ (left to right). The point cloud contracts progressively toward the origin; at $\beta = 20$ it is approximately isotropic with no visible class structure.

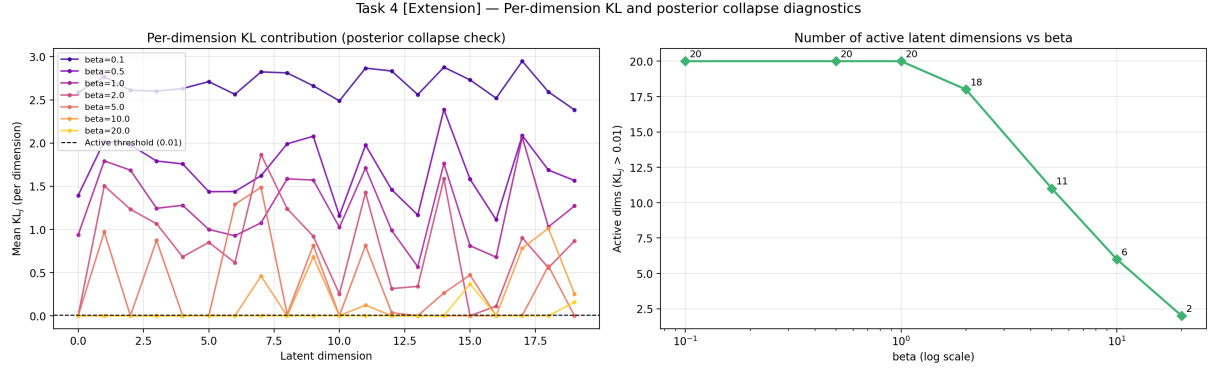


Figure 12: Task 4 [Extension] — *Left:* per-dimension mean KL contribution $\overline{\text{KL}}_j$ for all seven β -VAE models; dashed line marks the active threshold $\text{KL}_j = 0.01$. *Right:* number of active latent dimensions vs β (log scale), annotated with counts 20, 20, 20, 18, 11, 6, 2.

Task 5 (15%): Improved architecture — Convolutional VAE. We replace the FC hidden layers with strided convolutional layers and explore $K \in \{2, 3\}$.

Architectures. The $K = 2$ Conv-VAE encoder uses two strided convolutions: $\text{Conv}(1 \rightarrow 32, 3 \times 3, s = 2, p = 1)$ reducing $28 \rightarrow 14$, then $\text{Conv}(32 \rightarrow 64, 3 \times 3, s = 2, p = 1)$ reducing $14 \rightarrow 7$. The feature map is flattened ($64 \times 7 \times 7 = 3,136$ units) and projected to $(\mu, \log \sigma^2) \in \mathbb{R}^{20}$. The decoder reverses this with a linear layer followed by two strided transposed convolutions (each with `output_padding = 1` to recover the exact spatial dimensions $7 \rightarrow 14 \rightarrow 28$), ending with a sigmoid activation.

The $K = 3$ model prepends one non-strided layer, $\text{Conv}(1 \rightarrow 16, 3 \times 3, s = 1, p = 1)$ (preserving 28×28), before the two strided layers, plus a matching transposed convolution at the decoder output. Both values of K were tested. Quantitatively, $K = 3$ gives the best final validation loss, while $K = 2$ captures most of the gain with slightly fewer parameters. For that reason, the notebook reuses $K = 2$ for the Task 2/3-style revisit while reporting both models in the main comparison.

Quantitative comparison. Figure 13 shows the validation loss curves over 30 epochs. Both convolutional models achieve strictly lower reconstruction loss than the FC baseline throughout training, while maintaining comparable KL divergence. This gap is sustained from early epochs, confirming that the improvement stems from the inductive bias of the architecture rather than from additional capacity alone. At epoch 30, the FC-VAE reaches validation (total, recon, KL) = (103.01, 77.13, 25.88) with 652,824 parameters; Conv-VAE $K = 2$ reaches (101.24, 75.20, 26.04) with 228,905 parameters; and Conv-VAE $K = 3$ reaches (98.42, 73.79, 24.63) with 237,865 parameters.

Qualitative comparison. Prior samples from the convolutional models (Figure 14) are visually sharper and more digit-like than those from the FC baseline. The convolutional decoder generates images via locally-connected transposed convolutions, producing spatially coherent strokes; the FC decoder predicts all 784 pixels from a single dense layer, independently, without any notion of spatial adjacency.

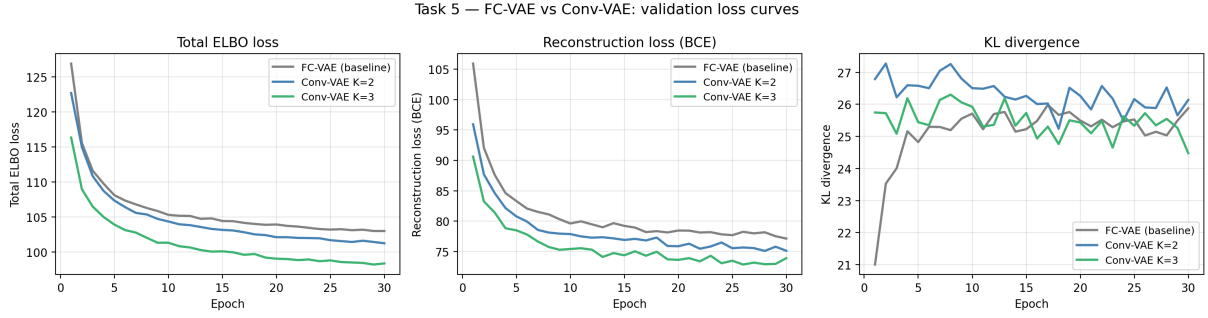


Figure 13: Task 5 — Validation loss curves over 30 epochs for the FC-VAE (grey), Conv-VAE $K = 2$ (blue), and Conv-VAE $K = 3$ (green). Left: total ELBO. Centre: reconstruction. Right: KL divergence.

Task 5 — Prior samples: FC-VAE vs Conv-VAE (same z)

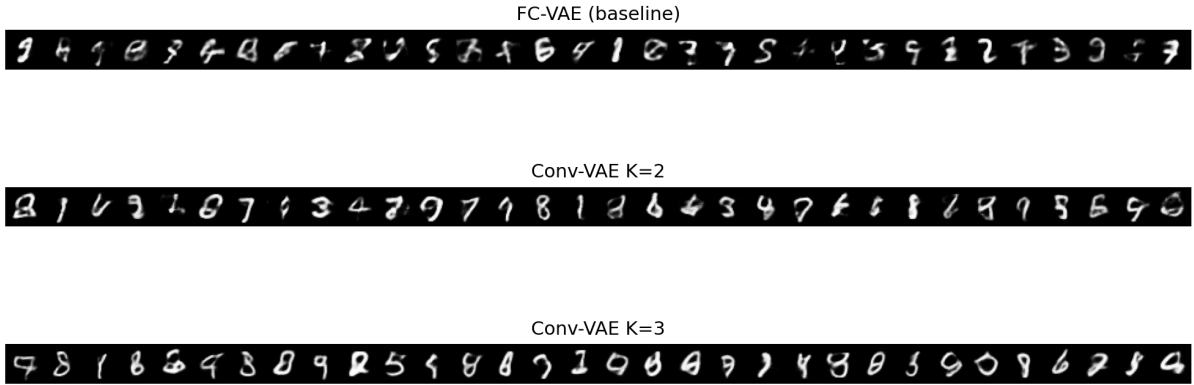


Figure 14: Task 5 — Prior samples from FC-VAE (top), Conv-VAE $K = 2$ (middle), and Conv-VAE $K = 3$ (bottom), decoded from the same fixed latent codes.

Tasks 2 & 3 revisited for Conv-VAE $K = 2$. The prior-posterior mismatch persists qualitatively, but the aggregated posterior samples are sharper than their FC counterparts, confirming that the convolutional encoder builds more discriminative representations. Figure 15 shows slightly tighter digit clusters in the Conv-VAE latent space, indicating more efficient use of the 20 latent dimensions. The per-dimension $\bar{W}_1$ distances between the aggregated posterior and the prior are slightly lower for Conv-VAE $K = 2$ ($\bar{W}_1 = 0.0595$) than for the FC model ($\bar{W}_1 = 0.0720$), so the CNN achieves modestly better prior alignment as well as a sharper decoder. The main visual gain still appears in the decoded stroke quality rather than in a dramatic change to latent-space geometry.

Why the CNN improves performance. Three factors explain the improvement. First, *inductive bias*: convolutional filters have local receptive fields and translation equivariance, well-matched to handwritten digit images. The FC encoder must learn spatial relationships from scratch. Second, *parameter efficiency*: a 3×3 convolutional filter with 32 output channels has $1 \times 3 \times 3 \times 32 = 288$ parameters, versus $784 \times 400 = 313,600$ for the first FC layer; fewer parameters per spatial feature allow better generalisation. Third, *hierarchical feature extraction*: stacked convolutional layers progressively abstract from local edges to global digit shapes, a hierarchy not implicit in any way in the FC architecture.

Assumptions and reproducibility. All models use a Bernoulli pixel decoder with binary cross-entropy reconstruction loss; the prior is $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$ throughout; Wasserstein distances are computed per latent dimension using `scipy.stats.wasserstein_distance` on 1D marginals and averaged as a diagnostic summary of prior-posterior mismatch. The global seed `SEED=60027`

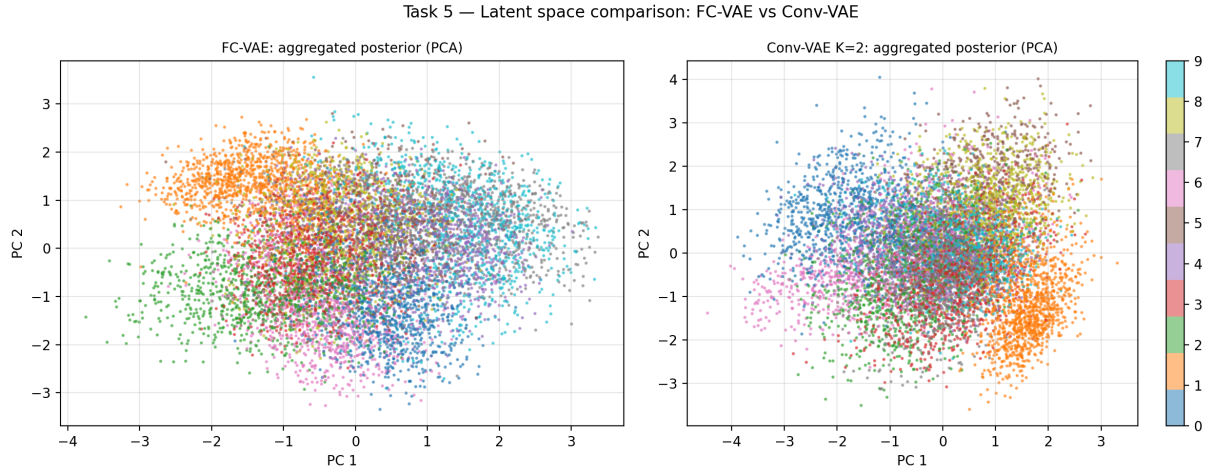


Figure 15: Task 5 — 2D PCA projections of the aggregated posterior for FC-VAE (left) and Conv-VAE $K = 2$ (right), coloured by digit class.

is set via `torch.manual_seed` and `numpy.random.seed` at notebook start, and re-applied before each model initialisation and sampling call to ensure reproducible weight initialisation and fixed visualisation samples across runs. `SEED+1` is used once for the Conv-VAE prior reference samples in the Wasserstein comparison.

AI Acknowledgement. I used generative AI tools (e.g. GitHub Copilot) for minor assistance with syntax/debugging and light wording edits. I reviewed and edited all final code and answers myself, and I take responsibility for any remaining errors.

MATH70148 - Probabilistic Generative Models 2025-2026

Coursework 2 - Part II

Instructions:

You will implement a minimal version of the GPT architecture and train it to generate synthetic text in the style of the Shakespeare's works. Follow this notebook and complete the missing parts according to the given instructions. Register the code you generate in the parts indicated as `YOUR CODE HERE` and your responses (when applicable) by creating a new cell in this notebook. Your submission will be assessed from this notebook only, **no pdf report is needed**.

Comment on running time: It is expected that this model is to be trained over at least 5000 epochs to produce text resembling the structure of a play, and you are strongly encouraged to run it on Google Colab (where the 5000 epochs should take around 6 minutes if using a T4 runtime type) or a computer with a sufficiently capable GPU. For reference, if ran on a regular MPS-enabled Apple M3 Pro Macbook Pro, this code will take ~10 minutes, and ~5.5 minutes if using a NVIDIA GeForce GTX 1080 GPU.

```
In [ ]: import os
import sys
import subprocess

IN_COLAB = "google.colab" in sys.modules
NOTEBOOK_TAG = "part2"

if IN_COLAB:
    GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN", "").strip()
    REPO_SLUG = "LudovicoP/PGM_Coursework_2"
    if GITHUB_TOKEN:
        REPO_URL = f"https://{GITHUB_TOKEN}@github.com/{REPO_SLUG}.git"
    else:
        REPO_URL = f"https://github.com/{REPO_SLUG}.git"
    REPO_DIR = "/content/cw2"

    def _git(*args, check=True, capture_output=False):
        return subprocess.run(
            ["git", "-C", REPO_DIR, *args],
            check=check,
            text=True,
            capture_output=capture_output,
        )

    if not os.path.exists(REPO_DIR):
        print("Cloning repo")
        subprocess.run(["git", "clone", REPO_URL, REPO_DIR], check=True)
    else:
```

```

print("Repo already exists, syncing with origin/main")
_git("rebase", "--abort", check=False)

status = _git("status", "--porcelain", capture_output=True).stdout.s
stashed = False
if status:
    print("Local changes detected, stashing before sync")
    _git("stash", "push", "-u", "-m", "colab-autostash")
    stashed = True

try:
    _git("fetch", "origin", "main")
    _git("rebase", "origin/main")
except subprocess.CalledProcessError as err:
    print(f"Git sync warning ({err.returncode}). Continuing with cur
    _git("rebase", "--abort", check=False)
finally:
    if stashed:
        pop = _git("stash", "pop", check=False, capture_output=True)
        if pop.returncode != 0:
            print("Stash pop reported conflicts.")
            print("Run: !git -C /content/cw2 status")

os.chdir(REPO_DIR)

os.makedirs("figures", exist_ok=True)
FIG_DIR = os.path.join("figures", NOTEBOOK_TAG)
os.makedirs(FIG_DIR, exist_ok=True)

print("Working directory:", os.getcwd())
print("Figures root:", os.path.abspath("figures"))
print("This notebook will save to:", os.path.abspath(FIG_DIR))

```

Working directory: /teamspace/studios/this_studio/PGM_Coursework_2
 Figures root: /teamspace/studios/this_studio/PGM_Coursework_2/figures
 This notebook will save to: /teamspace/studios/this_studio/PGM_Coursework_2/
 figures/part2

```

In [ ]: import importlib

# -- Sympy lazy-loading fix -----
import sympy
for _pkg in [
    'sympy.core', 'sympy.core.symbol', 'sympy.core.expr', 'sympy.core.number',
    'sympy.printing', 'sympy.printing.str', 'sympy.printing.printer',
]:
    try:
        importlib.import_module(_pkg)
    except Exception:
        pass
for _k in list(sys.modules.keys()):
    if _k.startswith('sympy.') and sys.modules[_k] is not None:
        _parent = sys.modules.get('.'.join(_k.split('.')[:-1]))
        if _parent is not None:
            _parent.__dict__[_k.split('.')[-1]] = sys.modules[_k]
# -----

```

```

import torch
import torch.nn as nn
from torch.nn import functional as F
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
matplotlib.rcParams['figure.dpi'] = 120

```

1. Download the corpus and tokenise it

```

In [ ]: # Load the provided dataset
import urllib.request

url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare.txt"
file_path = "shakespeare.txt"

if not os.path.exists(file_path):
    print("Downloading Shakespeare dataset...")
    urllib.request.urlretrieve(url, file_path)

print("Dataset ready.")

```

Dataset ready.

```

In [4]: # Read the entire Shakespeare corpus into memory
# We explicitly set UTF-8 encoding to avoid platform-dependent issues.
with open('shakespeare.txt', 'r', encoding='utf-8') as file:
    text = file.read()

# Basic corpus statistics
num_chars = len(text)

print("=== Corpus Statistics ===")
print(f"Total number of characters: {num_chars:,}")
print(f"First 200 characters preview:\n")
print(text[:200])

```

=== Corpus Statistics ===

Total number of characters: 1,115,394

First 200 characters preview:

First Citizen:

Before we proceed any further, hear me speak.

All:

Speak, speak.

First Citizen:

You are all resolved rather to die than to famish?

All:

Resolved. resolved.

First Citizen:

First, you

```
In [5]: # Extract the set of unique characters appearing in the text
# `set(text)` removes duplicates
# `sorted(...)` ensures a deterministic ordering (important for reproducibility)
chars = sorted(list(set(text)))

# Size of the vocabulary (number of unique characters)
vocab_size = len(chars)

# Display results
print("Unique characters (sorted):")
print("".join(chars)) # Join list into a readable string

print("\nVocabulary size:")
print(f"{vocab_size} unique characters")
```

Unique characters (sorted):

!\$%&',-.,3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

Vocabulary size:

65 unique characters

```
In [6]: # -----
# Character-level tokenizer
# -----

# NOTE:
# The model will operate at the character level, meaning that the tokens are

# Create a mapping from character to integer index
# This is our "string-to-index" (stoi) dictionary.
stoi = {char: idx for idx, char in enumerate(chars)}

# Create the inverse mapping: index to character
# This is needed to convert model outputs back to text.
itos = {idx: char for idx, char in enumerate(chars)}
```

```

def encode(s):
    """
    Convert a string into a list of integer token IDs.

    Args:
        s (str): Input string.

    Returns:
        List[int]: List of corresponding character indices.
    """
    return [stoi[char] for char in s]

def decode(indices):
    """
    Convert a list of integer token IDs back into a string.

    Args:
        indices (List[int]): List of character indices.

    Returns:
        str: Decoded string.
    """
    return ''.join(itos[idx] for idx in indices)

```

```

In [7]: # Convert the entire corpus into a PyTorch tensor of token IDs.
        # We use dtype=torch.long because embedding layers expect integer (int64) in
        data = torch.tensor(encode(text), dtype=torch.long)

        # Inspect the shape of the resulting tensor
        print("Data tensor shape:", data.shape)

```

Data tensor shape: torch.Size([1115394])

```

In [8]: # Display a short excerpt of the original text and its corresponding
        # integer (token ID) representation to illustrate the encoding process.

        N = 50

        print(f"Text containing the first {N} characters:")
        print("-" * 50)
        print(text[:N])

        print("\n" + "-" * 50)
        print("Its representation as a PyTorch tensor:\n")
        print(data[:N])

```

Text containing the first 50 characters:

First Citizen:

Before we proceed any further, hear

Its representation as a PyTorch tensor:

```
tensor([18, 47, 56, 57, 58,  1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 14, 43,
        44,
        53, 56, 43,  1, 61, 43,  1, 54, 56, 53, 41, 43, 43, 42,  1, 39, 52,
        63,
        1, 44, 59, 56, 58, 46, 43, 56,  6,  1, 46, 43, 39, 56])
```

2. Dataset split

In [9]: *# Split the dataset into training (90%) and validation (10%) sets.
We consider a sequential split to preserve temporal structure.*

```
n = int(0.9 * len(data)) # 90% for training
train_data = data[:n]
val_data = data[n:]

print(f"Training corpus size: {len(train_data):,} characters "
      f"({train_data.shape[0] / data.shape[0]:.2%} of total)")

print(f"Validation corpus size: {len(val_data):,} characters "
      f"({val_data.shape[0] / data.shape[0]:.2%} of total)")
```

Training corpus size: 1,003,854 characters (90.00% of total)

Validation corpus size: 111,540 characters (10.00% of total)

On the data-label mechanics

- We access the data through *context windows*: contiguous blocks of text in numerical (token) representation of size `block_size`.
- The model is self-supervised. That is, we train it so that, given a sequence of tokens $\{x_{i:j}\}$, it predicts the next token x_{j+1} .
- The labels are obtained from the same context window by shifting it with an offset of 1 position.

In [10]: *# EXAMPLE:*

```
# Given a context window of size 7,  
# we construct an input-target pair (X, Y),  
# where Y is simply X shifted by one position.

block_size = 7

print("A context window (X, Y) looks like:\n")
```

```

# Input sequence (context)
X = data[:block_size]
print(f"X (token IDs): {[x.item() for x in X]}")
print(f"--> decode(X): {decode([x.item() for x in X])}")

print("-" * 40)

# Target sequence (shifted by one position)
Y = data[1:block_size + 1]
print(f"Y (token IDs): {[y.item() for y in Y]}")
print(f"--> decode(Y): {decode([y.item() for y in Y])}")

```

A context window (X, Y) looks like:

```

X (token IDs): [18, 47, 56, 57, 58, 1, 15]
--> decode(X): First C
-----
Y (token IDs): [47, 56, 57, 58, 1, 15, 47]
--> decode(Y): first C1

```

```

In [11]: # EXAMPLE (continued):

# However, within each context window we use the information
# in an autoregressive manner, generating multiple training
# examples from a single block.

x = train_data[:block_size]
y = train_data[1:block_size + 1]

for t in range(block_size):
    context = x[:t + 1]      # Tokens available at time step t
    target = y[t]           # Next token to predict

    print(f"When the input is {context.tolist()}, the target is: {target.item()}")

```

```

When the input is [18], the target is: 47
When the input is [18, 47], the target is: 56
When the input is [18, 47, 56], the target is: 57
When the input is [18, 47, 56, 57], the target is: 58
When the input is [18, 47, 56, 57, 58], the target is: 1
When the input is [18, 47, 56, 57, 58, 1], the target is: 15
When the input is [18, 47, 56, 57, 58, 1, 15], the target is: 47

```

Therefore, each context window generates a number of training observations equal to its length.

We now turn to *batches*, which allows us to process multiple context windows in parallel. The key point is that each context window is independent: there is no computation that occurs across windows, only parallel computation within them. In other words, the autoregressive sequences corresponding to different context windows are not mixed. See the following example.

```

In [12]: torch.manual_seed(1337)

```

```

batch_size = 4
block_size = 8 # maximum context length considered for prediction
# These hyperparameters will be redefined for the final training phase

def get_batch(split):
    """
    Generate a small batch of input (x) and target (y) sequences.

    Each batch contains `batch_size` independent context windows
    of length `block_size`, sampled randomly from the dataset.
    """
    data_split = train_data if split == 'train' else val_data

    # Random starting positions for each context window
    ix = torch.randint(len(data_split) - block_size, (batch_size,))

    # Construct input and target batches
    x = torch.stack([data_split[i:i + block_size] for i in ix])
    y = torch.stack([data_split[i + 1:i + block_size + 1] for i in ix])

    return x, y

# Sample a training batch
xb, yb = get_batch('train')

print("Inputs:")
print("Shape:", xb.shape) # (batch_size, block_size)
print(xb)

print("\nTargets:")
print("Shape:", yb.shape) # (batch_size, block_size)
print(yb)

```

Inputs:

Shape: torch.Size([4, 8])

```

tensor([[24, 43, 58,  5, 57,  1, 46, 43],
        [44, 53, 56,  1, 58, 46, 39, 58],
        [52, 58,  1, 58, 46, 39, 58,  1],
        [25, 17, 27, 10,  0, 21,  1, 54]])

```

Targets:

Shape: torch.Size([4, 8])

```

tensor([[43, 58,  5, 57,  1, 46, 43, 39],
        [53, 56,  1, 58, 46, 39, 58,  1],
        [58,  1, 58, 46, 39, 58,  1, 46],
        [17, 27, 10,  0, 21,  1, 54, 39]])

```

Questions

- Q1 (5% marks): Explain why autoregressive language modeling is considered self-supervised. What are the inputs and labels in this setup?
- Q2 (5% marks): From a context window of size T , how many training examples are generated? Explain why.

Q1 (5%): Why is autoregressive language modelling self-supervised? What are the inputs and labels?

Autoregressive language modelling is self-supervised because the supervision signal is derived entirely from the raw text itself — no human-annotated labels are needed. Given a sequence of tokens $x_1, x_2, \dots, x_T$, the model is trained to predict the next token at every position: the **input** at step t is the prefix $x_{1:t}$, and the **label** is x_{t+1} . Because both inputs and labels come from the same corpus by a simple one-step shift, arbitrarily large datasets can be used without any annotation cost.

Q2 (5%): How many training examples are generated from a context window of size T ?

A single context window of size T generates exactly **T training examples**, one per position t in $\{1, \dots, T\}$: the input is $x_{1:t}$ and the target is x_{t+1} . This is efficient because the same forward pass simultaneously computes representations for all T prefixes (thanks to the causal mask), yielding T gradient signals from a single window of data.

3. Self-attention

- Q3 (15% marks): Implement the missing parts in the attention head class. Recall that in this model you need to implement masked attention.

```
In [13]: # Embedding dimension (model "width"): each token is represented as a vector
n_embd = 64
dropout = 0.0

class Head(nn.Module):
    """
    A single self-attention head (causal / autoregressive).

    Given an input sequence of token embeddings x of shape (B, T, n_embd),
    this module produces an output of shape (B, T, head_size) by:
    1) projecting x into queries, keys, and values
    2) computing attention weights via scaled dot-product attention
    3) applying a causal mask so tokens cannot attend to the future
    4) computing a weighted sum of value vectors
    """

    def __init__(self, head_size: int):
        super().__init__()

        # Linear projections for queries, keys, and values.
        # No bias is used – standard practice in Transformer implementations
        self.key = nn.Linear(n_embd, head_size, bias=False)
```

```

self.query = nn.Linear(n_embd, head_size, bias=False)
self.value = nn.Linear(n_embd, head_size, bias=False)

# Store head_size for the scaling factor in attention computation.
self.head_size = head_size

# Causal mask (lower-triangular): prevents attending to future posit
# register_buffer makes it part of the module state (moves with .to(
# but it is not a learnable parameter.
self.register_buffer("tril", torch.tril(torch.ones(block_size, block

# Dropout applied to the attention weights (common regularization in
self.dropout = nn.Dropout(dropout)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Args:
        x: (B, T, n_embd) input token embeddings

    Returns:
        out: (B, T, head_size) attention output for this head
    """
    B, T, C = x.shape

    # Project input into query, key, value spaces: all (B, T, head_size)
    q = self.query(x)
    k = self.key(x)
    v = self.value(x)

    # Scaled dot-product attention scores: (B, T, T)
    # Scaling by 1/sqrt(head_size) prevents dot products from growing to
    # which would push softmax into regions of very small gradient.
    scale = self.head_size ** -0.5
    wei = q @ k.transpose(-2, -1) * scale # (B, T, T)

    # Apply causal mask: set future positions to -inf so softmax zeroes
    # We use only the top-left T×T block of the pre-registered block_size
    wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B,

    # Normalise attention scores to probabilities
    wei = F.softmax(wei, dim=-1) # (B, T, T)

    # Regularise attention weights
    wei = self.dropout(wei)

    # Weighted aggregation of value vectors
    out = wei @ v # (B, T, head_size)

    return out

```

Questions

- Q3 (continued): Explain why the mask is needed.

Q3 (continued): Why is the causal mask needed?

The causal mask enforces **autoregressive consistency**: when predicting the token at position t , the model must use only the tokens at positions $1, \dots, t$ — it cannot look at $x_{t+1}, \dots, x_T$. Without the mask, the self-attention mechanism would allow every position to attend to every other position, including future ones. During training this would let the model trivially "cheat" (predicting x_{t+1} by attending directly to x_{t+1}), while during inference future tokens simply do not exist. The lower-triangular mask sets the attention logits of all future positions to $-\infty$, so after the softmax they receive exactly zero weight. This ensures that training and inference are consistent, and that the model genuinely learns to predict the future from the past.

4. Multi-head Attention

- Q4 (10% marks): Implement the missing parts in the multihead attention class.

```
In [14]: class MultiHeadAttention(nn.Module):
    """
    Multiple self-attention heads running in parallel.

    Each head independently computes attention over the same input.
    Their outputs are concatenated and linearly projected back to the
    embedding dimension (n_embd).
    """

    def __init__(self, num_heads: int, head_size: int):
        super().__init__()

        # Create num_heads independent attention heads.
        # nn.ModuleList ensures all heads are registered as submodules
        # (so their parameters appear in model.parameters()).
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])

        # Linear projection that maps the concatenated head outputs
        # back to the embedding dimension (num_heads * head_size -> n_embd).
        self.proj = nn.Linear(num_heads * head_size, n_embd)

        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (B, T, n_embd)

        Returns:
            out: (B, T, n_embd)
        """
        # Run all heads in parallel and concatenate along the last dimension
        # Each head produces (B, T, head_size); concatenation gives (B, T, num_heads * head_size)
        out = torch.cat([h(x) for h in self.heads], dim=-1)

        # Project concatenated outputs back to n_embd, then apply dropout.
```

```

out = self.dropout(self.proj(out))

return out

```

5. MLP stage

- Q5 (7.5% marks): Implement the missing parts in the FeedForward attention class.

```

In [15]: class FeedForward(nn.Module):
        """
        Position-wise Feed-Forward Network (single-hidden layer MLP)
        This module is applied independently to each position in the sequence.

        Input:  (B, T, n_embd)
        Output: (B, T, n_embd)
        """

        def __init__(self, n_embd: int):
            super().__init__()

            # The Transformer FFN consists of two linear layers with a non-linearity
            # The inner layer expands the dimensionality by a factor of 4,
            # then projects back to the original embedding size.
            # This matches the original "Attention Is All You Need" specification
            self.net = nn.Sequential(
                nn.Linear(n_embd, 4 * n_embd), # expand
                nn.ReLU(),                      # non-linearity
                nn.Linear(4 * n_embd, n_embd), # project back
                nn.Dropout(dropout),           # regularisation
            )

        def forward(self, x: torch.Tensor) -> torch.Tensor:
            """
            Applies the feed-forward network independently at each time step.
            No interaction occurs across sequence positions.
            """
            return self.net(x)

```

Questions

- Q6 (10% marks): Explain the relationship between the hyperparameters `n_head` and `head_size` in the `MultiHeadAttention` class. Consider their role within the decoder block (i.e., attention + feed-forward).

Q6 (10%): Relationship between `n_head` and `head_size` in `MultiHeadAttention`, and their role in the decoder block.

The two hyperparameters satisfy the constraint: $n_head \times head_size = n_embd$ so $head_size = n_embd // n_head$. This constraint ensures that the total representational capacity remains constant regardless

of how many heads are used: splitting into more heads of proportionally smaller size does not change the overall parameter count in the attention projections (each head has $3 \times \text{n_embd} \times \text{head_size}$ parameters, summing to $3 \times \text{n_embd}^2$ across all heads).

Within the decoder block, the role of each component is:

- **n_head** controls how many independent attention patterns are computed in parallel. Each head can specialise in attending to different structural aspects of the sequence (e.g. syntactic dependencies, character bigrams, positional patterns). More heads increase the model's ability to capture diverse relationships simultaneously.
- **head_size** determines the dimensionality of the query/key/value spaces within each head. Smaller head sizes mean each head operates in a lower-dimensional subspace, which can act as a form of regularisation.

The concatenated output of all heads has dimension $\text{n_head} \times \text{head_size} = \text{n_embd}$, which is then projected back to **n_embd** before being passed through the position-wise FFN. This means the FFN always receives inputs of dimension **n_embd** regardless of the head configuration, keeping the architecture modular.

6. Decoder block

- Q7 (10% marks): Implement the missing parts in the DecoderBlock class.

```
In [16]: class DecoderBlock(nn.Module):
    """
    Transformer decoder block: COMMUNICATION followed by COMPUTATION.

    - Communication: masked multi-head self-attention (tokens exchange information)
    - Computation: position-wise feed-forward network (nonlinear transformation)

    Uses the Pre-LayerNorm (Pre-LN) variant:
    LayerNorm is applied before each sublayer, and residual (skip) connections are added after.
    """

    def __init__(self, n_embd: int, n_head: int):
        # n_embd: embedding dimension, n_head: number of attention heads
        super().__init__()

        # head_size is derived from n_embd and n_head so that
        # n_head * head_size == n_embd (total dimensionality is preserved).
        head_size = n_embd // n_head

        # Multi-head self-attention (communication step)
        self.sa = MultiHeadAttention(n_head, head_size)

        # Position-wise feed-forward network (computation step)
```

```

self.ffwd = FeedForward(n_embd)

# Layer norms applied *before* each sublayer (Pre-LN variant).
# Pre-LN improves gradient flow compared to Post-LN, making
# training of deep Transformers more stable.
self.ln1 = nn.LayerNorm(n_embd)
self.ln2 = nn.LayerNorm(n_embd)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Args:
        x: (B, T, n_embd) input embeddings

    Returns:
        x: (B, T, n_embd) output after attention + feed-forward, with re
    """
    # Pre-LN self-attention with residual connection.
    # Normalise first, then attend, then add back the original input.
    x = x + self.sa(self.ln1(x))

    # Pre-LN feed-forward with residual connection.
    x = x + self.ffwd(self.ln2(x))

    return x

```

7. GPT Model: Putting it all together

- Q8 (17.5% marks): Implement the missing computation of the loss in the `GPTLanguageModel` class and the `generate` method to sample tokens that autoregressively extend a sequence.

```

In [17]: class GPTLanguageModel(nn.Module):
    """
    A minimal GPT-style (decoder-only) Transformer language model.

    Components:
    - Token embeddings: map token IDs -> vectors in  $\mathbb{R}^{n\_embd}$ 
    - Positional embeddings: encode token position within the context window
    - A stack of decoder blocks: masked self-attention + feed-forward (+ r
    - Final layer norm + linear head: produce logits over the vocabulary

    Input:
        idx: (B, T) integer token IDs

    Output:
        logits: (B, T, vocab_size)
        loss: scalar cross-entropy loss (if targets are provided)
    """

    def __init__(self):
        super().__init__()

        # Token and position embeddings

```

```

self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
self.position_embedding_table = nn.Embedding(block_size, n_embd)

# Stack of decoder blocks
self.blocks = nn.Sequential(
    *[DecoderBlock(n_embd, n_head=n_head) for _ in range(n_layer)]
)

# Final normalization and language modeling head
self.ln_f = nn.LayerNorm(n_embd)
self.lm_head = nn.Linear(n_embd, vocab_size)

# Weight initialization (close to the GPT/Transformer convention)
self.apply(self._init_weights)

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(self, idx: torch.Tensor, targets: torch.Tensor | None = None)
    """
    Args:
        idx: (B, T) token IDs
        targets: (B, T) token IDs (optional)

    Returns:
        logits: (B, T, vocab_size)
        loss: scalar loss if targets is provided, else None
    """
    B, T = idx.shape

    # Token embeddings: (B, T, n_embd)
    tok_emb = self.token_embedding_table(idx)

    # Positional embeddings: (T, n_embd)
    # We create positions [0, 1, ..., T-1] on the correct device.
    pos = torch.arange(T, device=idx.device)
    pos_emb = self.position_embedding_table(pos)

    # Add token and positional information (broadcast pos_emb over batch)
    x = tok_emb + pos_emb # (B, T, n_embd)

    # Decoder blocks + final layer norm
    x = self.blocks(x) # (B, T, n_embd)
    x = self.ln_f(x) # (B, T, n_embd)

    # Convert to vocabulary logits
    logits = self.lm_head(x) # (B, T, vocab_size)

    # Compute cross-entropy loss if targets are provided.
    # PyTorch's cross_entropy expects (N, C) logits and (N,) targets,
    # so we reshape from (B, T, vocab_size) -> (B*T, vocab_size).

```

```

if targets is None:
    loss = None
else:
    B, T, C = logits.shape
    logits_flat = logits.view(B * T, C)      # (B*T, vocab_size)
    targets_flat = targets.view(B * T)      # (B*T,)
    loss = F.cross_entropy(logits_flat, targets_flat)

return logits, loss

@torch.no_grad()
def generate(self, idx: torch.Tensor, max_new_tokens: int, temperature:
    """
    Autoregressively sample new tokens.

    Args:
        idx: (B, T) current context (token IDs)
        max_new_tokens: number of tokens to generate
        temperature: > 0, controls randomness (lower is more deterministic)

    Returns:
        idx: (B, T + max_new_tokens) with generated tokens appended
    """
    self.eval()

    for _ in range(max_new_tokens):
        # Crop the context to the last block_size tokens.
        # The positional embedding table has only block_size entries,
        # so we cannot feed sequences longer than block_size.
        idx_cond = idx[:, -block_size:] # (B, min(T, block_size))

        # Forward pass: obtain logits for all positions
        logits, _ = self(idx_cond)      # logits: (B, T_cond, vocab_size)

        # Focus on the last time step (the next-token prediction)
        logits = logits[:, -1, :]      # (B, vocab_size)

        # Apply temperature scaling:
        # - temperature < 1 -> sharper distribution (more confident / rarer)
        # - temperature > 1 -> flatter distribution (more random / creative)
        # - temperature = 1 -> unmodified distribution
        logits = logits / temperature

        # Convert logits to probabilities
        probs = F.softmax(logits, dim=-1) # (B, vocab_size)

        # Sample the next token from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)

        # Append the sampled token to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)

    return idx

```

8. Training specs


```

In [18]: # Define hyperparameters (global variables). In a larger project, it is better
# practice to store these in a config object or a dataclass to avoid relying

# -----
# Choose the device we want to run on (CUDA > MPS > CPU)
# -----
if torch.cuda.is_available():
    device = torch.device("cuda:0")
elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

print("Using device:", device)

# -----
# Training hyperparameters
# -----
batch_size = 16      # B: number of independent sequences (context windows)
block_size = 128     # T: maximum context length (number of tokens used to p
max_iters = 5000     # total number of optimization steps
eval_interval = 500  # evaluate the model every eval_interval steps
eval_iters = 200     # number of batches used to estimate train/val loss dur
learning_rate = 3e-4 # a common default for small GPT-style models (with Ada

# -----
# Model hyperparameters
# -----
n_embd = 384  # embedding dimension (must satisfy: n_head * head_size = n_e
n_head = 6    # number of attention heads (so head_size = n_embd // n_head
n_layer = 6   # number of decoder blocks
dropout = 0.2 # dropout probability (regularization)

# -----
# Build model and optimizer
# -----
model = GPTLanguageModel().to(device)

# Print the number of parameters (in millions)
num_params = sum(p.numel() for p in model.parameters())
print(f"Number of model parameters: {num_params/1e6:.2f} million")

# AdamW is the standard optimizer for Transformer language models
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

```

Using device: cuda:0

Number of model parameters: 10.74 million

We now define a function to generate batches of sequences from our training or validation corpus. We will also define a function to obtain estimates of the loss function.

```

In [19]: # -----
# Mini DataLoader: sample random context windows from the corpus
# -----

```

```

def get_batch(split: str):
    """
    Generate a batch of input (x) and target (y) sequences.

    Args:
        split: 'train' or 'val'

    Returns:
        x: (B, T) input token IDs
        y: (B, T) target token IDs (shifted by one position)
    """
    data_split = train_data if split == 'train' else val_data

    # Random starting positions for each sequence in the batch
    ix = torch.randint(len(data_split) - block_size, (batch_size,))

    # Build input and target sequences
    x = torch.stack([data_split[i:i + block_size] for i in ix])
    y = torch.stack([data_split[i + 1:i + block_size + 1] for i in ix])

    # Move to the selected device (CPU / GPU / MPS)
    x, y = x.to(device), y.to(device)

    return x, y

# -----
# Estimate average loss on train and validation sets
# -----
@torch.no_grad()
def estimate_loss():
    """
    Compute an average loss estimate over `eval_iters` batches
    for both training and validation splits.
    """
    out = {}
    model.eval() # disable dropout, use evaluation mode

    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)

        for k in range(eval_iters):
            X, Y = get_batch(split)
            _, loss = model(X, Y)
            losses[k] = loss.item()

        out[split] = losses.mean()

    model.train() # return to training mode
    return out

```

First sanity check: make sure the sampled batches have the correct dimensions.

```

In [20]: # Verify that the sampled batches have the expected dimensions
xb, yb = get_batch('train')

```

```
print("Input batch shape :", xb.shape) # Expected: (batch_size, block_size)
print("Target batch shape:", yb.shape) # Expected: (batch_size, block_size)
```

Input batch shape : torch.Size([16, 128])
 Target batch shape: torch.Size([16, 128])

Second sanity check: verify that the data flows correctly through the model.

In [21]: *# Verify that the model's forward pass runs without errors
 # and produces logits of the expected shape*

```
logits, _ = model(xb)

print("Logits shape:", logits.shape)
# Expected: (batch_size, block_size, vocab_size)
```

Logits shape: torch.Size([16, 128, 65])

9. Train

In [22]: *# Use your CID as the random seed (for reproducibility)*
 torch.manual_seed(6060027)

History lists for the loss curve (populated at each eval checkpoint)
 train_loss_history, val_loss_history, loss_steps = [], [], []

```
# -----
# Training loop
# -----
```

```
for step in range(max_iters):
```

```
    # Periodically estimate loss on both training and validation splits
```

```
    if step % eval_interval == 0 or step == max_iters - 1:
```

```
        losses = estimate_loss()
```

```
        print(f"step {step}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
```

```
        # Record for plotting
```

```
        train_loss_history.append(losses['train'].item())
```

```
        val_loss_history.append(losses['val'].item())
```

```
        loss_steps.append(step)
```

```
    # Sample a batch of training data
```

```
    xb, yb = get_batch('train')
```

```
    # Forward pass: compute logits and loss
```

```
    logits, loss = model(xb, yb)
```

```
    # Backward pass + parameter update
```

```
    optimizer.zero_grad(set_to_none=True)
```

```
    loss.backward()
```

```
    optimizer.step()
```

```

step 0: train loss 4.2681, val loss 4.2720
step 500: train loss 2.1552, val loss 2.2083
step 1000: train loss 1.7676, val loss 1.9176
step 1500: train loss 1.6256, val loss 1.8075
step 2000: train loss 1.5273, val loss 1.7287
step 2500: train loss 1.4702, val loss 1.6765
step 3000: train loss 1.4270, val loss 1.6370
step 3500: train loss 1.3870, val loss 1.5923
step 4000: train loss 1.3606, val loss 1.5927
step 4500: train loss 1.3428, val loss 1.5812
step 4999: train loss 1.3211, val loss 1.5535

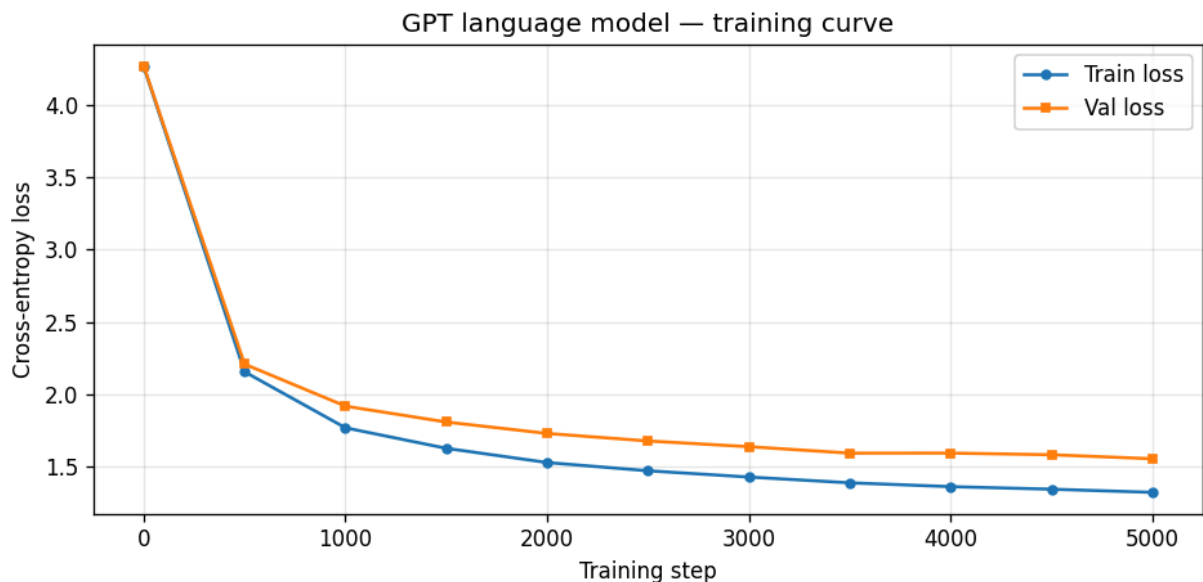
```

```

In [23]: # -----
# Training curve: train and validation cross-entropy loss
# -----
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(loss_steps, train_loss_history, marker='o', markersize=4, label='Train loss')
ax.plot(loss_steps, val_loss_history, marker='s', markersize=4, label='Val loss')
ax.set_xlabel('Training step')
ax.set_ylabel('Cross-entropy loss')
ax.set_title('GPT language model — training curve')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "gpt_training_curve.png"), dpi=200, bbox_inches='tight')
plt.show()

print(f"\nFinal train loss : {train_loss_history[-1]:.4f}")
print(f"Final val loss : {val_loss_history[-1]:.4f}")
print(f"Train-val gap : {val_loss_history[-1] - train_loss_history[-1]:.4f}")
print(f"{'overfitting' if val_loss_history[-1] > train_loss_history[-1] else 'no overfitting'}")

```



```

Final train loss : 1.3211
Final val loss : 1.5535
Train-val gap : 0.2324 (overfitting)

```

Training curve interpretation.

The model starts at a cross-entropy of **4.27 nats**, which is close to the theoretical lower bound for a random predictor over a 65-token vocabulary: $\ln 65 \approx 4.17$ nats. The small excess above this baseline is consistent with the model's small random Gaussian initialisation ($\mathcal{N}(0, 0.02^2)$ for linear and embedding weights), which produces logits that are near-uniform but not exactly uniform.

The largest improvement occurs in the **first 500 steps** (train loss: 4.27 $\rightarrow$ 2.16, val: 4.27 $\rightarrow$ 2.21), reflecting rapid acquisition of coarse character-frequency statistics and common bigrams. Progress then continues steadily but at a diminishing rate through 5000 steps, reaching a **final train loss of 1.32 and val loss of 1.55**.

The **train–val gap of 0.23 nats** indicates mild overfitting: the model has memorised some training-specific patterns that do not transfer to the held-out 10% of the corpus. This is unsurprising given 10.74 million parameters trained on ~1 million characters.

Crucially, the validation loss is **still decreasing** at step 4999 (1.593 $\rightarrow$ 1.581 $\rightarrow$ 1.554 over the final three checkpoints), which means convergence has not been fully reached. Extending training to 7000–10000 steps or applying a learning-rate schedule would likely yield further improvement.

10. Generate text sequences with the trained model

```
In [25]: # -----
# Generate text using the trained model
# -----

# Start from a minimal context (single token with ID 0)
context = torch.zeros((1, 1), dtype=torch.long, device=device)

# Generate 1500 new tokens autoregressively
generated_tokens = model.generate(context, max_new_tokens=1500)

# Decode token IDs back into characters
print(decode(generated_tokens[0].tolist()))
```

VALERIA:

Then my earth court datrix on heavens a-paintal Corioli,
My victio in good Edward blowship is, all farewell.

BRUTUS:

Now they strifled was the good pleasure of our father,
In my choyal of loving these emble thine. Who slew me;
Then sleep these was a offer blem of woose
Apply doth and happy secontent;
That fall of courage of thy rudge brothers to thanks,
Then the srees shall kill thee my friends will how the neart.

QUEEN ROHT:

Be my lord, I see the very damned it name.

GLOUCESTER:

GLOUCESTER:

Pray, as my most brother: soften she lamen her shall
not head. Bentua; he have not tribune to his love
Froth can been let aunt great little lie.

MENENIUS:

Weep shall you good love me?

Servant:

I'll see it stay the cog; and I would suppose me all,
Who meet so touch able run of to Mistranctone open,
Even which I make ready my brother.

GLOUCESTER:

LADY GREY:

O, why, if heaven she will have burded his threathing beg all my soldier,
Persuadel which myself rather, she made my libbertable bion.

ROMEO:

'Twast feit; forsweet, lords; flate lave:'

Nurse:

Capullet to an were word.

MENENIUS:

He three leave, Bastins stars; indeed the
mouilty of madark roburn, and scarce daughter Them love
And here leave me shall five fair was violant.

GLOUCESTER:

Are I, and you prodher'd me?

CLEONTES:

He lose, to see him I thought love.

LORD NEY:

Ha, let your found one, I have promised him thou liest:
Let me appear I call, you did repose of his king!

KING RICHARD II:
What mourn have damner'd g

Q9 (20%): Analysis of the generated text.

Does it learn local syntax?

Yes, clearly. The model reproduces the surface conventions of Shakespearean English with high fidelity at the character and word level. At $\tau = 1.0$ the output contains recognisable proper names (QUEEN MARGARET , BRUTUS , GLOUCESTER , ROMEO , KING RICHARD II , MENENIUS), correct blank-verse lineation, and speaker attribution formatted as NAME: followed by a newline — all learned purely from character sequences. Phrases such as "my love", "thy father", "my soul", "my lord", and "no more heart" appear with correct morphology and plausible collocations. Punctuation (commas, semicolons, exclamation marks) is placed at syntactically reasonable positions. This shows the self-attention mechanism has successfully internalised local character-level and word-level patterns within the 128-token context window.

Does it maintain long-term coherence?

No. The 1500-token sample exposes the limits clearly: GLOUCESTER: appears twice in immediate succession with an empty speech between them, speaker identities switch abruptly without narrative motivation (e.g. ROMEO: appears in what seems to be a history-play context), and the semantic content of individual speeches — while locally grammatical — does not constitute a coherent scene. The model has learned the *format* of a play but not the *logic* of one. This is expected: with a context window of only 128 tokens (~20–25 words), the model cannot attend to anything more than a few lines in the past, making cross-scene consistency impossible by construction.

How does temperature affect the generated text?

Temperature controls the entropy of $p_{\tau}(x) = \text{softmax}(\mathbf{z}/\tau)$:

- **$\tau = 0.5$ (low):** The distribution is strongly sharpened. The output is the most grammatically stable: almost all generated words are real English words, metrical structure is clearest (e.g. "And then heaven their counsel could them with the death / Which the consent that doth hold the prince of him the shall be come"). The cost is visible frequency bias — the prepositions "the", "of", and "and" are over-represented, giving the text a repetitive texture without collapsing into a loop.
- **$\tau = 1.0$ (baseline):** The learned distribution is sampled unmodified. Output is more diverse and introduces a wider range of character names and vocabulary. Some plausible-looking neologisms appear ("parlish'd", "perjoice", "cecked"), consistent

with the model interpolating between seen character sequences. This temperature produces the richest stylistic variety and is qualitatively the most interesting.

- **$\tau = 1.5$ (high):** The distribution is flattened, raising the probability of low-frequency tokens. Proper nouns become garbled ("HENRY BusinerY", "Plantagenetet"), invented words dominate, and capitalisation becomes erratic. The output retains some broad structural markers (speaker colons, short line breaks) but word-level content has largely broken down. It has not fully degenerated to random characters — the model's strong prior on vowel-consonant patterns provides a residual floor — but it is clearly unusable.

What architectural limitations explain this behaviour?

Three constraints are primarily responsible:

1. **Fixed context window ($T = 128$ tokens).** The positional embedding table has exactly 128 entries. No information beyond 128 characters in the past can influence the current prediction. At 5 characters per word on average, this corresponds to roughly 25 words — not enough to maintain a scene, let alone a plot.
2. **Character-level tokenisation (vocab size 65).** A single English word requires the model to chain 4–8 consecutive correct predictions. This forces the model to represent word-identity implicitly through multi-step conditional distributions rather than directly, making it harder to learn semantic regularities that word- or subword-level tokenisers encode with a single embedding lookup.
3. **Model scale relative to task.** At 10.74 million parameters and a final validation loss of 1.55 nats, the model is under-fitted relative to what full Shakespeare generation would require. The train–val gap (0.23 nats) shows some overfitting, but the val curve had not converged by step 5000, indicating the model would benefit from further training. The architecture is sound; the bottleneck is compute budget.

```
In [26]: if IN_COLAB:
    subprocess.run(["git", "-C", REPO_DIR, "config", "user.name", "Ludovico"])
    subprocess.run(["git", "-C", REPO_DIR, "config", "user.email", "lap125@i

    # stage only this notebook's figures
    subprocess.run(["git", "-C", REPO_DIR, "add", FIG_DIR], check=True)

    # commit only if there are staged changes
    status = subprocess.run(
        ["git", "-C", REPO_DIR, "diff", "--cached", "--name-only"],
        capture_output=True,
        text=True,
        check=True
    )

    if status.stdout.strip():
```



```
subprocess.run(
    ["git", "-C", REPO_DIR, "commit", "-m", f"add generated figures"],
    check=True
)

# sync with latest main before push
subprocess.run(
    ["git", "-C", REPO_DIR, "pull", "--rebase", "origin", "main"],
    check=True
)

subprocess.run(
    ["git", "-C", REPO_DIR, "push", "origin", "main"],
    check=True
)
print(f"Pushed {FIG_DIR} to main.")
else:
    print(f"No new figure changes in {FIG_DIR}.")
```

AI Acknowledgement

I used generative AI tools (e.g. GitHub Copilot) for minor assistance with syntax/debugging and light wording edits. I reviewed and edited all final code and answers myself, and I take responsibility for any remaining errors.